

PRACTICAL WEEK8

TASK1

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    // malloc() - allocates memory without initialization
    int *arr1 = (int *)malloc(3 * sizeof(int));

    if (arr1 == NULL)
    {
        printf("Memory allocation using malloc failed.\n");
        return 1;
    }

    for (i = 0; i < 3; i++)
    {
        arr1[i] = (i + 1) * 10;
    }

    printf("Memory allocated using malloc():\n");
    for (i = 0; i < 3; i++)
    {
        printf("%d ", arr1[i]);
    }
    printf("\n");

    // calloc() - allocates and initializes memory to zero
    int *arr2 = (int *)calloc(3, sizeof(int));

    if (arr2 == NULL)
    {
        printf("Memory allocation using calloc failed.\n");
        free(arr1);
        return 1;
    }
}
```

```

}

printf("\nMemory allocated using calloc():\n");
for (i = 0; i < 3; i++)
{
    printf("%d ", arr2[i]);
}
printf("\n");

// realloc() - changes the size of allocated memory
arr1 = (int *)realloc(arr1, 5 * sizeof(int));

if (arr1 == NULL)
{
    printf("Memory reallocation failed.\n");
    free(arr2);
    return 1;
}

arr1[3] = 40;
arr1[4] = 50;

printf("\nAfter realloc(), arr1 contains:\n");
for (i = 0; i < 5; i++)
{
    printf("%d ", arr1[i]);
}
printf("\n");

// free() - releases allocated memory
free(arr1);
free(arr2);

printf("\nMemory released using free().\n");

```

```

laasya@laasya:~/week8$ nano task1.c
laasya@laasya:~/week8$ gcc task1.c -o task1
laasya@laasya:~/week8$ valgrind --leak-check=full ./task1
==2019== Memcheck, a memory error detector
==2019== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==2019== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==2019== Command: ./task1
==2019==
Memory allocated using malloc():
10 20 30

Memory allocated using calloc():
0 0 0

After realloc(), arr1 contains:
10 20 30 40 50

Memory released using free().
==2019==
==2019== HEAP SUMMARY:
==2019==    in use at exit: 12 bytes in 1 blocks
==2019==    total heap usage: 4 allocs, 3 frees, 1,068 bytes allocated
==2019==
==2019== 12 bytes in 1 blocks are definitely lost in loss record 1 of 1
==2019==    at 0x484D953: calloc (in /usr/libexec/valgrind/vgpreload_memchec
k-amd64-linux.so)
==2019==    by 0x1092E1: main (in /home/laasya/week8/task1)
==2019==
==2019== LEAK SUMMARY:
==2019==    definitely lost: 12 bytes in 1 blocks
==2019==    indirectly lost: 0 bytes in 0 blocks
==2019==    possibly lost: 0 bytes in 0 blocks
==2019==    still reachable: 0 bytes in 0 blocks
==2019==    suppressed: 0 bytes in 0 blocks
==2019==
==2019== For lists of detected and suppressed errors, rerun with: -s
==2019== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
laasya@laasya:~/week8$ =

```

TASK2

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (10 * 1024 * 1024)

int main()
{
    char *data;
    pid_t pid;

    // Allocate 10 MB of memory
    data = (char *)malloc(SIZE);

    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }

    // Write to all pages so the memory is actually allocated
    for (size_t i = 0; i < SIZE; i++)
    {
        data[i] = 'A';
    }

    printf("Parent: allocated and initialized 10 MB.\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        free(data);
        return 1;
    }
}
```

```

    // Child process
    printf("\nChild process created.\n");
    printf("Child PID: %d\n", getpid());

    printf("Child: reading shared memory...\n");
    printf("data[0] = %c\n", data[0]);

    printf("Child: modifying memory...\n");

    // Modify the memory
    for (size_t i = 0; i < SIZE; i++)
    {
        data[i] = 'B';
    }

    printf("Child: memory modification completed.\n");
    printf("data[0] = %c\n", data[0]);

    free(data);
    exit(0);
}
else
{
    // Parent process
    printf("\nParent: waiting for child...\n");

    wait(NULL);

    printf("Parent: child has finished.\n");
    printf("Parent: data[0] = %c\n", data[0]);

    free(data);
}

return 0;

```

```

laasya@laasya:~/week8$ nano task2.c
laasya@laasya:~/week8$ gcc task2.c -o task2
laasya@laasya:~/week8$ ./task2
Parent: allocated and initialized 10 MB.
Parent PID: 2043

Parent: waiting for child...

Child process created.
Child PID: 2044
Child: reading shared memory...
data[0] = A
Child: modifying memory...
Child: memory modification completed.
data[0] = B
Parent: child has finished.
Parent: data[0] = A
laasya@laasya:~/week8$ ps -o pid,ppid,rss,vsz,cmd
  PID   PPID  RSS   VSZ  CMD
   685    680 5368   6080 -bash
  2047    685 4052   8244 ps -o pid,ppid,rss,vsz,cmd
laasya@laasya:~/week8$

```

```
laasya@laasya: ~  
laasya@laasya:~$ ps -o pid,ppid,rss,vsz,cmd | grep task2  
2093    2076    2100    4100 grep --color=auto task2  
laasya@laasya:~$ cat /proc/12346/status | grep -E "VmSize|VmRSS"  
cat: /proc/12346/status: No such file or directory  
laasya@laasya:~$
```